

Founding Engineer - Pre-Interview Task

Why this exists, and how it's structured

This mirrors the actual shape of the job: go from raw student data → a model that adapts → something a teacher could use on a real device.

A note on AI tools: use Claude, GPT, Copilot, whatever you normally use — we're not going to pretend otherwise, and pretending it doesn't exist would be a bad filter anyway. What we're actually testing is whether you understand what got built, can extend it under pressure, and can defend the decisions in it. That's why this has two parts: a take-home, and a mandatory 30–45 minute live round based on it. **A submission without the live round isn't considered.** That live round, not the written code, is what we're actually grading.

Time budget for the take-home: aim for 4–6 focused hours.

Submission: a GitHub repo with a README, plus the write-up in Part 3. We'll schedule the live round within a few days of submission.

Part 0 — Get your own dataset (don't skip this)

You need real or realistic student-response data to work with — `student_id`, `skill_id`, `correct`, `timestamp`, `response_time_ms` (or close to it). Two options, your choice:

1. **Find a public education dataset** and adapt it — e.g. ASSISTments, Kaggle's student performance/quiz-response sets, or similar open knowledge-tracing datasets. Real data is messier than anything we could script for you, and you will end up with a different subset with different quirks — which is exactly the point.
2. **Generate your own synthetic dataset**, seeded with something specific to you (your name, email, or similar) so it's not identical to anyone else's. Deliberately introduce a few realistic imperfections yourself — missing timestamps, duplicate submissions, an implausible response pattern or two — and document what you introduced and why.

Part 1 — The core problem

You have a student's history of responses (correct/incorrect, with timestamps) on a specific skill. Build something that:

1. Given that history, estimates how likely the student is to answer correctly the *next* time they're tested on that skill.
2. Updates that estimate as each new response comes in — it should behave differently for a student who's clearly improving vs. one who's plateaued vs. one who's guessing.

3. Uses that estimate to recommend what to do next — an easier question, a harder one, or a different skill entirely.

We're deliberately not naming the class of technique this calls for. Figuring out what kind of problem this actually is, picking a defensible approach, and being able to justify why you didn't pick something else, is part of what's being evaluated — not whether you can look up a known term and reproduce a tutorial. Any approach is fair game as long as you understand what it assumes and where it breaks.

Part 2 — Ship it (full-stack)

1. Wrap Part 1 in a small API with at least:
 - `POST /response` — submit an answer, get back an updated estimate + next recommendation
 - `GET /student/{id}/estimate` — current per-skill estimate snapshot
2. Build a minimal client (web or mobile) that hits this API.
3. Handle offline/low-connectivity behavior for at least one real failure mode: queuing responses when offline and syncing without duplication, or graceful degradation with clear "not synced" states.

Part 3 — Write-up (half a page, no more)

1. What did you notice in your dataset that needed handling, and what did you do about it?
2. What approach did you pick for the core estimation problem, and what would break first with real classroom data (60 kids, guessing, messy patterns)?
3. What's the one thing you'd change about your offline/sync approach for a real 2G/3G government-school network?
4. What did you deliberately not build, and why?

Part 4 — Live round (this is the actual assignment)

30–45 minutes, screen-share, your code in front of you. This round is not scripted or shared in advance in any form — everything you're asked will be based on your specific submission and put to you live, on the spot. Expect to:

- Walk us through a decision you made and defend why you didn't take an alternative.
- Modify or extend your own code live, in response to a scenario we give you during the call.
- Reason through a small hypothetical by hand, out loud, with no IDE — using your own approach, not a memorized one.